The Store sobre K3s gestionado con Ansible

Tema 5 · Despliegue y gestión del clúster de Kubernetes

Grupo 6 · Nicolás Arancibia Carabajal · Lautaro Paletta · Agustín Ronda

Redes de Información · ITBA · 2C 2026

Agenda

1. Problemática: cómo se opera hoy The Store
2. Solución: diseño, arquitectura de red y tecnologías
3. Demo: los cinco casos de uso de la pre-entrega
4. Resultados, limitaciones y conclusiones

Problemática

The Store hoy

Tienda online con cinco microservicios:

- **ui** · Java (Spring Boot)
- **catalog** · Go
- **cart** · Java (Spring Boot)
- **orders** · Java (Spring Boot)
- **checkout** · Node.js (NestJS)

Se despliega con `local.sh`:

1. crea un clúster de **Kind** en una sola máquina,
2. carga las imágenes con `kind load`,
3. aplica los manifiestos.

Pensado para desarrollar, no para operar.

Problemática y objetivo

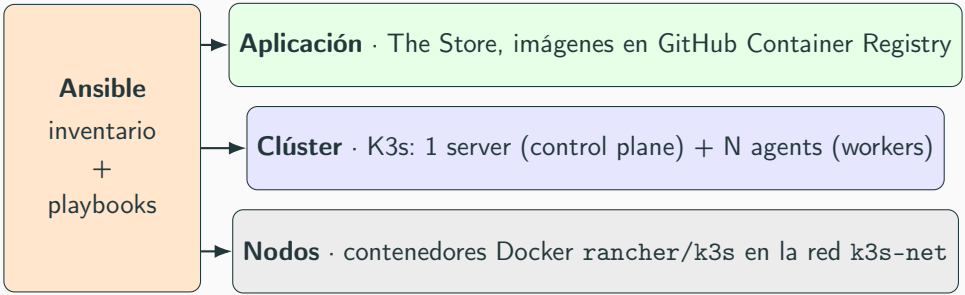
- Cada operación sobre el clúster se hace **a mano** y depende de quién la ejecute.
- No se pueden **agregar ni quitar nodos**: hay que borrar el clúster y recrearlo, con la tienda fuera de servicio.
- Ante un pico de tráfico (Black Friday, Navidad) no hay forma rápida de **sumar capacidad**.

Objetivo

Un clúster que cualquier integrante del equipo pueda **crear**, **ampliar**, **reducir** y **reconstruir** desde cero, de forma automatizada y repetible.

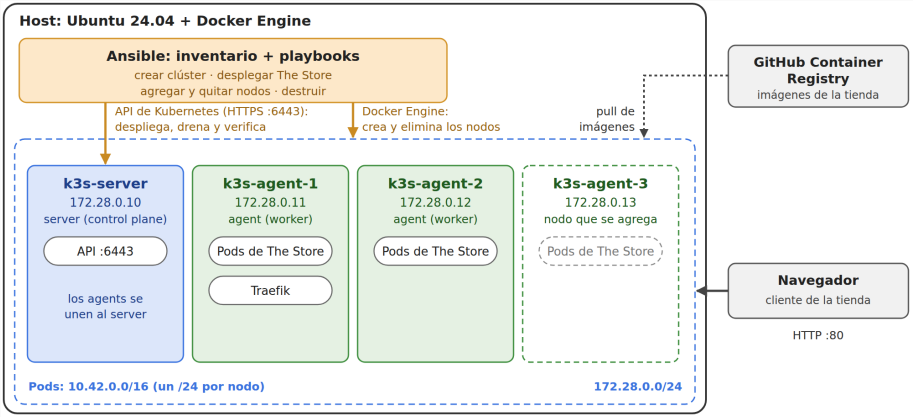
Solución

Diseño en tres capas



Ansible crea los nodos, instala el clúster y despliega la aplicación.
Desde ahí, Kubernetes mantiene la tienda corriendo y reparte los pods entre los nodos.

Arquitectura de red

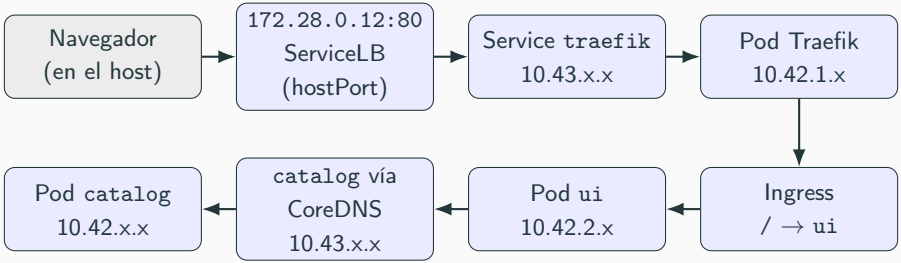


`k3s-net` 172.28.0.0/24 (gw .1) · Pods 10.42.0.0/16 (un /24 por nodo) · Services 10.43.0.0/16

Flujos de red

Origen → destino	**Protocolo**	**Para qué**
Ansible → Docker Engine	API Docker (socket Unix)	crea y elimina los nodos
Ansible → `k3s-server`	HTTPS · TCP 6443	despliega, drena y verifica
agents → `k3s-server`	HTTPS · TCP 6443 + token	unión al clúster
pod ↔ pod (entre nodos)	VXLAN · UDP 8472 (Flannel)	red de pods 10.42.0.0/16
nodos → `ghcr.io`	HTTPS · TCP 443 (NAT del host)	descarga de imágenes
navegador → cualquier nodo	HTTP · TCP 80	ServiceLB → Traefik → ui

Recorrido de un request



Si el pod destino está en otro nodo, el paquete viaja encapsulado en VXLAN sobre `k3s-net`.

Decisiones técnicas

Ansible compara el estado actual con el deseado y cambia solo la diferencia. Re-ejecutar no rompe nada y agregar un nodo no toca a los demás.

Nodos como contenedores arrancan en segundos y consumen una fracción de una VM. El rol `docker_node` está aislado: se puede cambiar por VMs.

Registry (GHCR) cada nodo descarga las imágenes por su cuenta; un nodo nuevo no depende de que le copien nada (reemplaza a `kind load`).

Traefik + ServiceLB vienen con K3s y publican el puerto 80 en cada nodo, sin instalar ni exponer ingress-nginx aparte.

Ansible: inventario, roles y playbooks

```
inventory/hosts.yml
  k3s_server:
    k3s-server  172.28.0.10
  k3s_agents:
    k3s-agent-1 172.28.0.11
    k3s-agent-2 172.28.0.12

roles/
  docker_node/  crea la "máquina"
  k3s/          API, kubeconfig, Ready
  the_store/    manifiestos
```

Playbook	Caso
`create-cluster.yml`	1 · Reproducibilidad
`deploy-store.yml`	2 · Validación
`add-nodes.yml`	3 · Capacidad
`remove-node.yml`	4 · Mantenimiento
`destroy.yml`	5 · Reconstrucción

Colecciones: `community.docker` y `kubernetes.core`.

Alternativas consideradas

Alternativa	**Qué ofrece**	**Por qué no**
kops en AWS	Máquinas, red y clúster; escalado y backup integrados	Costo, un solo proveedor, Ansible pierde su rol
Minikube	Un comando crea todo	SO y red propios: nada que diseñar ni automatizar
Kind / k3d	Nodos en Docker, como el nuestro	Kind no agrega ni quita nodos; k3d lo hace con sus propios comandos
VMs (Multipass o nube)	Lo más parecido a producción	Minutos por VM, $\geq$ 2 GB de RAM por nodo, costo en la nube

Demo

Scope del POC: cinco casos de uso

#	Caso	Criterio de éxito
1	Reproducibilidad	3 nodos Ready; la 2ª ejecución no cambia nada
2	Validación	Pods Running repartidos entre nodos; la tienda abre
3	Capacidad	Solo cambia el nodo nuevo; 4 nodos Ready
4	Mantenimiento	El nodo se va; sus pods corren en otros; la tienda sigue abriendo
5	Reconstrucción	Destroy no deja nada; el rebuild llega al estado de 1 y 2

Caso 1 · Reproducibilidad: levantar el clúster desde cero

Escenario: host con Docker, sin nada del proyecto. Inventario: 1 server + 2 agents.

```
ansible-playbook create-cluster.yml     # crea red, volúmenes y 3 nodos
ansible-playbook create-cluster.yml     # segunda vez
docker ps
kubectl get nodes -o wide
```

Criterio: 3 contenedores, 3 nodos `Ready`; la 1ª ejecución con cambios en cada nodo y la 2ª con `changed=0`.

Caso 2 · Validación: desplegar The Store

Escenario: clúster del caso 1, sin la aplicación.

```
ansible-playbook deploy-store.yml
kubectl get pods -n the-store -o wide
# navegador: http://172.28.0.11  y  http://172.28.0.12
```

Criterio: todos los pods `Running`, repartidos entre los nodos, y la tienda abre en el navegador.

Caso 3 · Capacidad: sumar un nodo

Escenario: clúster del caso 2 con la tienda funcionando.

```
# inventory/hosts.yml, dentro de k3s_agents:
#   k3s-agent-3:
#     node_ip: 172.28.0.13
ansible-playbook add-nodes.yml
kubectl get nodes
```

Criterio: solo `k3s-agent-3` tiene cambios; 4 nodos `Ready`.

Caso 4 · Mantenimiento: retirar un nodo

Escenario: clúster de cuatro nodos del caso 3.

```
ansible-playbook remove-node.yml -e node=k3s-agent-1
# después: sacar k3s-agent-1 de inventory/hosts.yml
kubectl get nodes
kubectl get pods -n the-store -o wide
docker ps
```

Criterio: el nodo no aparece en `kubectl get nodes` ni en `docker ps`, sus pods corren en otros nodos y la tienda sigue abriendo.

Caso 5 · Reconstrucción: destruir y volver a crear

Escenario: clúster funcionando con la tienda desplegada.

```
ansible-playbook destroy.yml
docker ps -a; docker network ls; docker volume ls
ansible-playbook create-cluster.yml
ansible-playbook deploy-store.yml
```

Criterio: después del destroy no queda nada del proyecto; después del rebuild, el clúster y la tienda quedan como al final de los casos 1 y 2.

Cierre

Resultados

	Caso	Evidencia
✓	1 · Reproducibilidad	3 nodos Ready · `changed=0` en la 2ª ejecución
✓	2 · Validación	Pods repartidos · tienda accesible por cualquier nodo
✓	3 · Capacidad	Solo cambió el nodo nuevo · 4 nodos Ready
✓	4 · Mantenimiento	Pods reubicados · nodo eliminado · tienda en línea
✓	5 · Reconstrucción	Sin residuos · mismo estado tras el rebuild

Limitaciones y trabajo futuro

Limitaciones del POC

- Una réplica por servicio: drenar su nodo corta la tienda unos segundos.
- Un solo control plane: sin alta disponibilidad.
- Un nodo nuevo puede no recibir pods al principio (localidad de imágenes).
- `k3s-net` es local al host: un cliente externo necesita un balanceador.

Cómo seguiría

- Réplicas + PodDisruptionBudget por servicio.
- Tres servers con etcd embebido.
- Reemplazar `docker_node` por un rol que cree VMs.
- Balanceador delante de los nodos.

Conclusiones

- El clúster pasó de ser un entorno armado a mano a uno que se **crea, amplía, reduce y reconstruye** con un comando.
- El **inventario** es la fuente de verdad: escalar es editar un archivo.
- La separación nodos / clúster / aplicación permite cambiar el proveedor de nodos sin reescribir el resto.

¿Preguntas?

Código y how-to

https://github.com/NickArancibia/tpe-redes

Respaldo · K3s dentro de contenedores

- Imagen `rancher/k3s:v1.33.4-k3s1`, contenedores **privilegiados** (K3s administra iptables y cgroups de los pods).
- `tmpfs` en `/run` y `/var/run`; volumen `<nodo>-data` en `/var/lib/rancher/k3s`.
- Server con `-tls-san=172.28.0.10` y taint `node-role.kubernetes.io/control-plane:NoSchedule`.
- Agents con `K3S_URL=https://172.28.0.10:6443` y `K3S_TOKEN`.
- Ansible copia el kubeconfig del server y lo reescribe para apuntar a la IP del server.

Respaldo · Capturas de la demo

TODO: una captura por criterio de éxito, tomadas en el ensayo final.